Dream QA

The team where we would love to work

Manuel de la Peña
QA Team Lead, Liferay Cloud
@mdelapenya

Castilla-La Mancha

About me

2018 - QA Team Lead @ Liferay Cloud, Inc.

2017 - Sr. Software Engineer

Core Infrastructure Team

Automation Engineer

2014 - GDG Toledo Organiser

2012 - Software Engineer

2011 - Support Engineer @ Liferay, Inc.

2010 - Java & Liferay Consultant @ Indra

2007 - Java Consultant @ VASS

2003 - IT Support @ JCCM & ACLM

2006 - 2012 Computers Science in Management Degree

2014- 2017 Master Degree: Research in Software Engineering and Computer Systems

Liferay Inc.

Liferay

Digital Experience Platform

- Enterprise Portal
- Web Experience
- Mobile Experience
- Documents
- Collaboration
- Forms & Workflow
- Integration
- Development Platform

Liferay

Commerce

- Product Catalog
- Commerce Experience
- Customer Service Portal
- Commerce Intelligence

Liferay

Analytics Cloud

- Single Customer View
- User Segments
- Web & Asset Analytics
- Customer Intelligence

Liferay

DXP Cloud

Enterprise PaaS Tailored for Liferay DXP

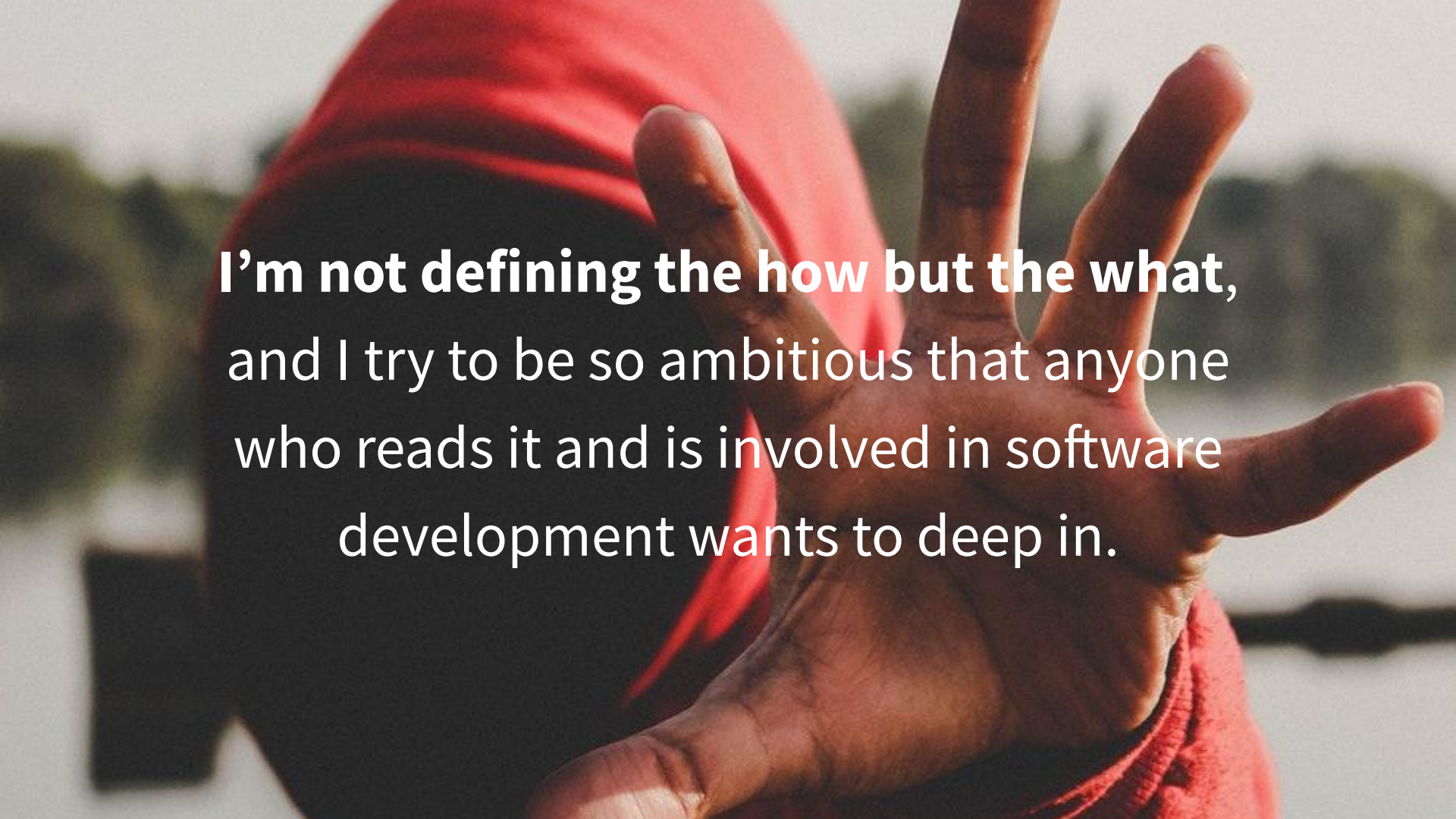
Focus on what matters

Deliver with confidence

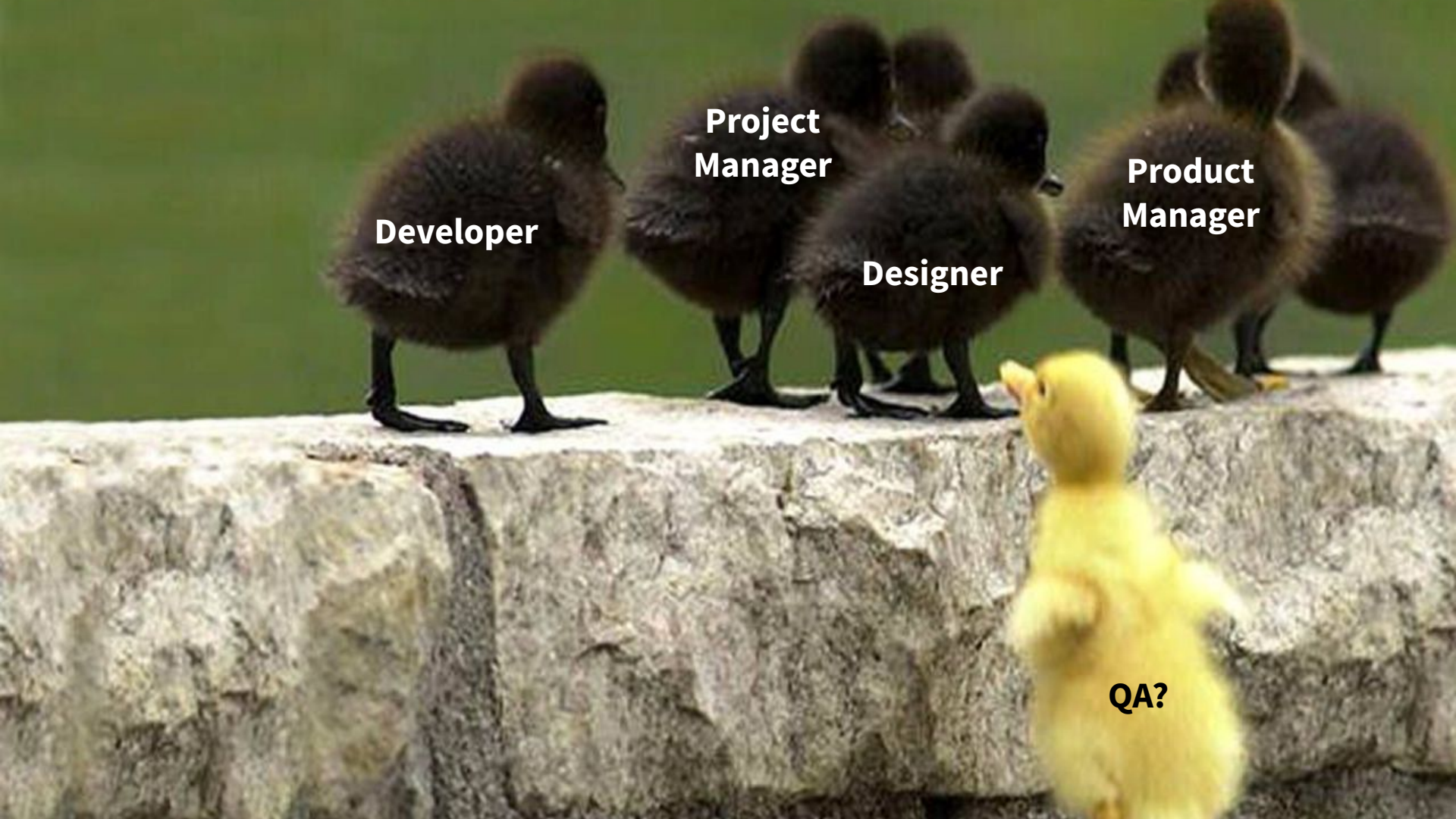
Evolve faster

As a QA Team Lead

1. Spread the culture of Quality
2. Enable Continuous Integration
3. Define pipelines across portfolio
4. Evolve CI platform
5. Evangelize about the benefits of well-written tests
6. Create tools improving Dev Experience
7. Release engineering (branching, tagging, artifacts definition)
8. Enable continuous delivery



I’m not defining the how but the what, and I try to be so ambitious that anyone who reads it and is involved in software development wants to deep in.


Developer
Project Manager
Designer
Product Manager
QA?

Testing

Quality ~~Assurance~~ *Assistance*

Define a Quality standard

Products

1

Define specific quality gates for any individual project conforming the products we build.

Processes

2

Pay attention to all tasks needed to design, implement, verify, release, deploy and operate the products we build.

Persons

3

Look for the right skills, promoting responsibility and ownership on daily tasks .

Unique definition of Quality across teams

Different teams could have opposing opinions about the same feature or bug fix.

The QA Process

Ultimate goal of the QA Process

“We want the QA process to be the same as the Development process”

The cross-functional QA process runs alongside the Dev process, not at the end of it, affecting the second process in all stages.

Creating the process is a shared responsibility

“Everybody is responsible for assuming and assuring the Quality of his/her work”

From Design to Operations, everybody knows about the importance of each part of the process, and they must ensure its own part.

Automate ‘Em All… :(

“Quality does not mean automating test cases”

- Automation is not a goal, is a tool
- Playing Devil’s Advocate: *If automation is a goal, then we should automate everything, which is impossible*.
  - **Infinite** scenarios in terms of possible paths and values in a feature under tests (FUT)
- Let’s cover the **most critical** features for the user with automation
  - Identify with Product Management the critical scenarios
  - Run critical tests in pull requests to reduce build time
  - Add manual/exploratory testing to cover the rest of the paths

Writing tests is good, but...

“Quality does not simply mean writing tests”

There is a lot of value in writing tests, but it’s not the single activity that guarantees Quality.

- Dependencies management
- Versioning (SEMVER)
- Branching strategy
- Release management
- Metrics gathering
- Open collaboration with other teams, as Design, or Product/Project Management.
- Spreading knowledge across teams.
- See Annexe later on.

Save time and money

“Do not postpone writing tests to the end of the process”

A bug found in production is super expensive in time and money. Let’s **try to break the feature** the soonest, leveraging test automation in all its forms: unit, integration, functional...

The QA Team

Gain a deep knowledge about the product

“We are a reference about how the product is expected to work”

We can discern between a bug or a feature, and for that reason, we have to be part of the rest of the phases of the development process, from Design to Deployment.

Shared responsibility

“We are not responsible for having the 100% of the test suite passing”

- The entire team is responsible to do so.
- We can also assist in finding the uncovered features in the test suite, increasing **feature coverage**, by the addition of more tests.
- We can raise a hand to stop machines and put the entire team to work on a failure in the test suite, addressing it to the specific product team and assisting them in the fix.

First-class citizens in the process

“We participate in defining the Definition of Done (DoD)”

Is “test passing” enough to continue in the pipeline?. We can assist any other team in defining the DoD for a task, assisting them in defining the quality gates that it needs.

Root-cause analysis

“We are able to identify the root cause and even send a fix for it”

- we can **reproduce the failure** and point to the commit which is the culprit of the failure, so the development team can fix it.
- we can reproduce the failure and **fix the line, block or feature affected** by a bug, if possible.
- we can reproduce the failure and update or remove a failing regression test, when needed.

Involves a **deep understanding of the codebase** or a very good set of **root-cause** analysis skills.

No more static docs

“We generate documentation in the form of automated test cases”

They represent the most valid, live documentation of the product.

- **Most valid**: they are versioned alongside the production code, so we could track when and why it changed.
- **live**: it can be automated, indicating the % of features passing in the test suite, which supports metrics, DoD, threshold definitions, etc.

The QA Manifesto

The QA Manifesto

In Liferay Cloud we prefer...

1. unit or integration tests, *over* functional (UI) tests.
2. BDD style for writing test scenarios, *over* any other type of internal documentation.
3. developers who write tests, *over* traditional testers who test.
4. testers who code, *over* traditional tester who write tests.
5. exploratory testing, *over* automating all possible test cases.
6. local checks and verifications of code, *over* online inspection of submitted code.
7. feature coverage, *over* code coverage.
8. confirming that requirements are not broken, *over* checking just the happy path.

QA Manifesto: #1

“Unit or Integration tests *over* (functional) UI tests”

- UI tests are extremely low and difficult to maintain. Unit and Integration are cheaper
- UI tests are usually maintained by the QA team, which is an error IMHO
- In an agile environment, where the code (UI) changes so frequently, they get usually broken
- The automated execution of UI tests take hours in the bests scenarios, so they are run very few.
- The addition of one UI tests usually adds minutes to the build, so it does not scale.

QA Manifesto: #2

“BDD style for writing tests *over* any other internal documentation”

- Gerkhin is a high level language to describe how the software should work
- Given / When / Then / And pattern
- Involve product managers in writing test scenarios using plain English is a must

QA Manifesto: #3

“Developers who write tests *over* traditional testers who test”

- In the past, traditional testers did manual testing in a repeated manner
- If we empower the developers with the ability of writing and maintaining the functional tests, then they are responsibles of having them green

QA Manifesto: #4

“Testers who code *over* traditional testers who write tests”

- In the past, traditional testers wrote functional tests scripts
- We want them to participate in coding a feature, from design to implementation

QA Manifesto: #5

“Exploratory Testing *over* automating all possible scenarios”

- How many possible scenarios are there in a feature? Infinite
  - Possible values
  - Different inputs combination
- Automate critical paths and perform exploratory testing over the rest of the paths
- Exploratory Testing (ET) is a learning process with the goal of becoming a domain expert
- From ET experience, we can decide to automate a specific new path discovered during exploration

QA Manifesto: #6

“Local checks and verifications of code *over* online inspections”

- Code is not merged directly: it’s send via a pull request
- Github is great, and reviewing pull requests online is cool...
- But that’s not ideal
- We want to download the code, locally verify its behaviour in a running instance of the software, and then continue or rollback the submission.

QA Manifesto: #7

“Feature coverage *over* Code coverage”

- Code coverage is a trap!
  - Tests that executes 100% of the code but without any assertion → 100% coverage
- Feature coverage is achieved in combination with BDD-style (see #2).
- We can decide whether to release or not based on thresholds around feature coverage

QA Manifesto: #8

“Confirming the requirements are not broken *over* confirming the happy path”

- Code without tests is broken by default, and if it works:
  - It works just for the happy path
  - It works by chance
- It’s important trying to demonstrate that the feature is not broken

Annexes

Best practices we (‘d love to) follow

Practice 1

“Involve Product Managers in writing test scenarios”

- PMs are real stakeholders of the features to implement, as they are the one of the best interfaces with the users and the market.
- Given-When-Then pattern
- BDD allows to use plain English to define test scenarios, so using Gerkhin in any of its implementations (Ruby, Java, NodeJS…) is a really good choice.

Practice 2

“Pair Review is mandatory when submitting code”

- A pull-request is mandatory to merge code into any project of the product, so we advise against directly merging into the master branch
- This pair review of the code ensures more quality on a feature, and also adds an intermediate layer to execute automated tests.
- We want to avoid silo-ed developers or rockstars that directly commit into the master branch

Practice 3

“Define pipelines representing each project, based on quality gates”

- A quality gate is a step in the pipeline
  - Compile / Build / Lint
  - Execute unit tests
  - Artifact generation / Artifact publication
  - Notifications (Slack, Email, Database)
- The successive quality gates represent the pipeline
- Automate the pipeline, and execute it with any code submission
  - Define bifurcations based on pull-requests or environments

Practice 4

“Define shared thresholds for releasability”

- Leverage the quality gates defined at pipeline level, like % of failed tests, as a threshold.
- PM defines the lower number of the threshold
- The product team defines the higher number.
- The range between them is used as tolerance for the metrics defined by the threshold (i.e % of functional tests, number of stories in Done)
    - Return a feature/task back if the threshold is not satisfied
- Create dashboards to improve the experience of reading the thresholds

Practice 5

“Measure and use those metrics along with Project Managers”

- Traditional static analysis of code: code complexity, findbugs, etc.
- Other interesting metrics:
  - The Running Tested Features (RTF)
  - Defects Found in Production / Escaped Defects
  - Defect Categories
  - Defect Cycle Time
  - Number of fixed bugs vs non reproducible, duplicate, and won't fix.

Practice 6

“*Before continuous delivery*, be predictable about releases”

- We have not achieved CD… but we are working on it!
- Define clear expectations about when and how to release code
- SEMVER is important!
- Release Train
  - Avoid the proliferation of mini-releases which include just a few changes, every day.
  - Reduce unplanned work, coupling requester and releaser to a single, shared moment ⇒ expectations are clear!

Thanks!

List of Images

- Dream: https://unsplash.com/photos/qYanN54gIrI
- Evolution: https://www.myersfreelance.com/wp-content/uploads/Evolution-of-SEO.jpg
- Stop: https://unsplash.com/photos/asytdeogTDE
- Ducks: http://justice.cadbayuc.org/wp-content/uploads/2015/05/animals-ugly-duckling11.jpg
- Wall: https://unsplash.com/photos/_Zb9R71acuc
- Rivers: https://unsplash.com/photos/wrTAgjWJg4c
- Handshake: https://unsplash.com/photos/__Xm_3r_DtM
- Old book: https://unsplash.com/photos/_Drvb_c_72Y
- Library: https://unsplash.com/photos/sfL_QOnmy00
- Check Icon: https://png.pngtree.com/svg/20170220/check_mark_903447.png
- Fusion: https://areajugones.sport.es/wp-content/uploads/2017/04/fusione-fredda-dragon-ball.jpg